1. Project Overview

This document details the end-to-end deployment of a MERN (MongoDB, Express.js, React, Node.js) stack application to AWS, following DevOps best practices including containerization, container orchestration, CI/CD automation, and infrastructure-as-code principles.

1.1 Application Architecture

Component	Technology & Details
Frontend	React.js — served via Nginx on port 80
Backend	Node.js + Express.js API on port 5050
Database	MongoDB on port 27017
ETL Service	Python-based data processing container
Orchestration	K3s (lightweight Kubernetes) on EC2
CI/CD	GitHub Actions with automated build & deploy
Registry	Docker Hub (public images)
Cloud	AWS EC2 t2.micro — us-east-1 (N. Virginia)

1.2 Repository Structure

GitHub Repository: DaniyalFarrukh/devops-case

Path	Purpose
docker-compose.yml	Multi-container Docker Compose orchestration
frontend/Dockerfile	React app containerized with Nginx
backend/Dockerfile	Express.js API container
etl/Dockerfile	Python ETL service container
k8s/	Kubernetes manifests (Deployments, Services)
.github/workflows/	GitHub Actions CI/CD pipeline YAML

2. Infrastructure Setup

2.1 AWS EC2 Instance

A single EC2 instance was provisioned on AWS to host the entire MERN stack. The following specifications were used:

Parameter	Value
Instance Type	t2.micro (1 vCPU, 1 GB RAM)
Operating System	Ubuntu 22.04 LTS
Region	us-east-1 (N. Virginia)
Public IP	3.223.246.202
Storage	8 GB EBS gp2
Swap Added	2 GB (permanent, for K3s stability)

Why 2 GB Swap?

t2.micro has only 1 GB RAM. K3s + Docker + 3 running containers exceed this limit. A 2 GB swap file was added permanently (/swapfile) to prevent OOM kills during builds and runtime. Configured in /etc/fstab for persistence across reboots.

2.2 Security Group Configuration

The EC2 Security Group was configured to allow the following inbound traffic:

Port / Protocol	Purpose
22 (TCP)	SSH access for server management
80 (TCP)	HTTP — Frontend React app (Nginx)
5050 (TCP)	Backend Express.js API
27017 (TCP)	MongoDB (internal; restricted to VPC)
6443 (TCP)	K3s Kubernetes API server

2.3 Swap Memory Setup

The following commands were run to add and persist the swap file:

```
sudo fallocate -l 2G /swapfile
sudo chmod 600 /swapfile
sudo mkswap /swapfile
```

```
sudo swapon /swapfile
echo '/swapfile none swap sw 0 0' | sudo tee -a /etc/fstab
```

3. Containerization with Docker

3.1 Docker Installation

Docker Engine was installed on the EC2 instance using the official Docker installation script for Ubuntu:

```
curl -fsSL https://get.docker.com -o get-docker.sh
sudo sh get-docker.sh
sudo usermod -aG docker ubuntu
```

3.2 Docker Base Images Pre-pulled

To speed up builds on the resource-constrained t2.micro instance, all base images were pre-pulled before building:

- node:18-alpine — for frontend and backend builds
- nginx:alpine — for serving the React production build
- python:3.11-slim — for the ETL service
- mongo:6 — for the MongoDB database

3.3 Dockerfile Breakdown

Frontend (React + Nginx)

The frontend uses a multi-stage build to keep the final image small:

```
# Stage 1: Build React app
FROM node:18-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
RUN npm run build

# Stage 2: Serve with Nginx
FROM nginx:alpine
COPY --from=builder /app/dist /usr/share/nginx/html
EXPOSE 80
```

Backend (Express.js API)

```
FROM node:18-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install --production
COPY . .
EXPOSE 5050
CMD ["node", "server.js"]
```

ETL Service (Python)

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
CMD ["python", "main.py"]
```

3.4 Docker Compose

docker-compose.yml was used to orchestrate all services locally and during deployment. It defines the networking, environment variables, and startup order for all containers:

```
services:
  frontend:
    build: ./frontend
    ports: ["80:80"]
    depends_on: [backend]
  backend:
    build: ./backend
    ports: ["5050:5050"]
    environment:
      MONGO_URI: mongodb://mongo:27017/devopsdb
    depends_on: [mongo]
  mongo:
    image: mongo:6
    ports: ["27017:27017"]
  etl:
    build: ./etl
    depends_on: [mongo]
```

4. Container Orchestration with K3s

4.1 Why K3s?

K3s is a lightweight, production-grade Kubernetes distribution designed for resource-constrained environments. It was the ideal choice for a t2.micro instance because it consumes significantly less memory than full Kubernetes (K8s), while still providing the full Kubernetes API and feature set.

4.2 K3s Installation

```
curl -sfL https://get.k3s.io | sh -
sudo systemctl enable k3s
sudo systemctl start k3s
sudo kubectl get nodes   # verify node is Ready
```

4.3 Kubernetes Manifests

Kubernetes YAML manifests were created in the k8s/ directory for each service. Each manifest defines a Deployment (for pod management and restarts) and a Service (for internal/external networking).

Key Kubernetes Concepts Applied

- Deployments — ensure desired number of pod replicas are always running
- Services — expose pods internally (ClusterIP) or externally (NodePort)
- ConfigMaps / Env Variables — manage configuration per environment
- Resource Limits — defined to prevent any single container starving the node

Manifest File	Purpose
k8s/frontend-deployment.yaml	1 replica of React/Nginx pod, NodePort 80
k8s/backend-deployment.yaml	1 replica of Express API pod, NodePort 5050
k8s/mongo-deployment.yaml	MongoDB pod with persistent volume claim
k8s/etl-deployment.yaml	ETL Python pod, runs as a job/service

5. CI/CD Pipeline with GitHub Actions

5.1 Pipeline Overview

A GitHub Actions CI/CD pipeline was configured at .github/workflows/deploy.yml. The pipeline triggers automatically on every push to the main branch and consists of four sequential jobs:

Job #	Job Name & Description
Job 1	Lint — Runs ESLint on the frontend and backend source code
Job 2	Test — Executes unit/integration tests for the backend API
Job 3	Build & Push — Builds Docker images and pushes to Docker Hub
Job 4	Deploy — SSHs into EC2 and runs docker compose up -d to redeploy

5.2 Pipeline YAML Structure

```yaml
name: CI/CD Pipeline

on:
  push:
    branches: [main]

jobs:
  lint:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - run: npm install && npm run lint

  test:
    needs: lint
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - run: npm test

  build-push:
    needs: test
    runs-on: ubuntu-latest
    steps:
      - uses: docker/login-action@v2
        with:
          username: ${{ secrets.DOCKERHUB_USERNAME }}
          password: ${{ secrets.DOCKERHUB_TOKEN }}
      - run: docker compose build && docker compose push
```

```
deploy:
  needs: build-push
  runs-on: ubuntu-latest
  steps:
    - uses: appleboy/ssh-action@v1
      with:
        host: ${{ secrets.EC2_HOST }}
        key: ${{ secrets.EC2_SSH_KEY }}
        script: |
          cd ~/devops-case
          docker compose pull
          docker compose up -d
```

5.3 GitHub Secrets Configured

Secret Name	Description
DOCKERHUB_USERNAME	Docker Hub account username
DOCKERHUB_TOKEN	Docker Hub access token (not password)
EC2_HOST	EC2 public IP: 3.223.246.202
EC2_SSH_KEY	Private SSH key (.pem) for EC2 ubuntu user

6. Deployment Process — Step by Step

6.1 Initial Setup on EC2

1. SSH into EC2: ssh -i devops-key.pem ubuntu@3.223.246.202
2. Update system: sudo apt update && sudo apt upgrade -y
3. Add 2 GB swap (see Section 2.3)
4. Install Docker using the official get-docker.sh script
5. Install K3s using the official install script
6. Clone repository: git clone https://github.com/DaniyalFarrukh/devops-case.git
7. Pre-pull all Docker base images

6.2 Build and Run with Docker Compose

```
cd ~/devops-case
docker compose up -d --build
docker ps   # verify all containers are running
```

6.3 Verify Services

Service	Verification URL / Command
Frontend (React)	http://3.223.246.202 — should show the web UI
Backend API	http://3.223.246.202:5050/healthcheck — returns {status: ok}
MongoDB	docker exec -it <mongo-container> mongosh
All Containers	docker ps — all should show STATUS: Up

6.4 Kubernetes Deployment (K3s)

```
sudo kubectl apply -f k8s/
sudo kubectl get pods          # check all pods Running
sudo kubectl get services      # check NodePort assignments
```

7. Challenges & Solutions

Challenge	Solution Applied
OOM kills during Docker build on t2.micro	Added 2 GB permanent swap file — resolved all OOM errors
Docker builds timing out (115+ seconds)	Pre-pulled all base images; builds now complete reliably
K3s API server instability on 1 GB RAM	Swap memory stabilized K3s — node stays in Ready state
GitHub Actions deploy job SSH auth failure	Stored EC2 .pem private key as EC2_SSH_KEY GitHub Secret
'version' obsolete warning in docker-compose.yml	Removed the top-level version: attribute from the file
BuildKit internal job NotFound error	Cosmetic internal scheduling message; not an actual failure
Containers not restarting after EC2 reboot	Added restart: unless-stopped to all services in Compose

8. Security Considerations

- SSH access is key-based only — password authentication is disabled on EC2
- Docker Hub credentials stored as GitHub Secrets — never hardcoded in source
- MongoDB is not exposed publicly — bound to the internal Docker network only
- EC2 Security Group follows least-privilege — only required ports are open
- EC2_SSH_KEY stored as an encrypted GitHub Secret — not visible in logs
- Production environment variables (MONGO_URI, API keys) injected at runtime

9. Technology Stack Summary

React.js	Node.js / Express	MongoDB	Python (ETL)
Docker	Docker Compose	K3s Kubernetes	GitHub Actions
AWS EC2	Ubuntu 22.04	Nginx	Docker Hub

10. Conclusion

This project successfully demonstrates a complete DevOps deployment of a MERN stack application following industry best practices. The key achievements are:

- Full containerization of all application components using Docker and best-practice multi-stage Dockerfiles
- Container orchestration with K3s (lightweight Kubernetes) managing all workloads on a single EC2 node
- Automated CI/CD pipeline via GitHub Actions with lint, test, build, and deploy stages
- Live deployment accessible at http://3.223.246.202 (frontend) and http://3.223.246.202:5050/healthcheck (backend API)
- Resilient setup with swap memory, restart policies, and persistent configuration across reboots
- Security-conscious design with SSH key authentication, least-privilege Security Groups, and Secret management

GitHub Repository

https://github.com/DaniyalFarrukh/devops-case All Dockerfiles, Kubernetes manifests, CI/CD pipeline, and this documentation are available in the repository.